

ACTIVATION SPARSIFICATION IN TRANSFORMERS: THRESHOLD PLACEMENT, PRESSURE PLACEMENT, AND SPARSE EXECUTION

Anonymous authors

Paper under double-blind review

ABSTRACT

Activation sparsity can accelerate language-model inference when specialized kernels skip computation associated with zero activations. Existing methods obtain sparsity through different combinations of training-time pressure (P) and thresholding (T), but their relative and combined effects have not been isolated under a common setup. We study these interventions under matched pretraining conditions across Pythia 14M and 70M models. For each combination of P and T , we derive an architecture-dependent maximum achievable sparsity ($\mathcal{S}_{\text{model}}^{\text{max}}$), measure model-wide attained activation sparsity ($\mathcal{S}_{\text{model}}$) and the final inference latency under a specialized sparse kernel. Results points to four different findings. First, **aggressive combinations of interventions can extend the attainable sparsity range**. The higher-thresholding ($\kappa = 0.5$) broader-site pressure, T_7/P_{all} , reaches 27.5% and 40.6% model-wide sparsity at 14M and 70M, respectively 92% and 82% of the architectural ceilings, but at the validation loss cost of 0.62 and 1.12. Secondly, **sparsity and speedup increase together within recipes**. Although they correlate, not all sparsity translates into speedups. Exploiting it requires specialized kernel development: 70M required a further optimization step because the 14M implementation did not transfer efficiently. For example, the sparser 14M recipe T_7/P_{all} achieves 27.48% sparsity but is slower than T_4/P_{all} at 12.71% (0.473 versus 0.459 ms). Third, we **observe a phenomenon of sparsity spillover**: local pressure can causa a non-local response. At 14M, FFN-hidden (h) pressure increases sparsity at the unpressured attention-output site (z) by 10.7 percentage points relative to no pressure, and **the biggest latency gains come from bypassing them**. Also, pressuring h alone is less harmful to validation loss than global pressure. We hypothesize that some transformer sites are more susceptible to sparsification than others. Our main takeaway is that training interventions and deployment kernels must be co-designed so that the induced sparsity is both high and structurally exploitable at inference time.

1 INTRODUCTION

Activation sparsity can improve transformer inference efficiency only when two conditions hold: activations must be exactly zero, and those zeros must occur in structures that the execution kernel can skip profitably (Liu et al., 2024; Cetin et al., 2026). Training-time activation pressure adds a gradient component to induce smaller activation magnitudes, while thresholding maps small values into exact zeros. This creates two distinct design choices: (1) where to apply the pressure (P), and (2) where to threshold (T). Because a transformer block exposes several candidate activation sites, this placement problem is combinatorial by design. The central question is therefore not simply *how sparse can a transformer become?*, but *where should sparsity be induced so that it produces useful execution savings at an acceptable quality cost?*

Existing methods make different choices within this design space. TEAL applies post-hoc magnitude thresholding to pretrained models, ProSparse uses ℓ_1 pressure on feed-forward (FFN) activations during training, and Q-Sparse applies top- k sparsification to both FFN and attention activations (Liu et al., 2024; Song et al., 2024; Wang et al., 2024). Because these methods differ simultaneously in sparsification mechanism, intervention sites, training setup, and execution implementation,

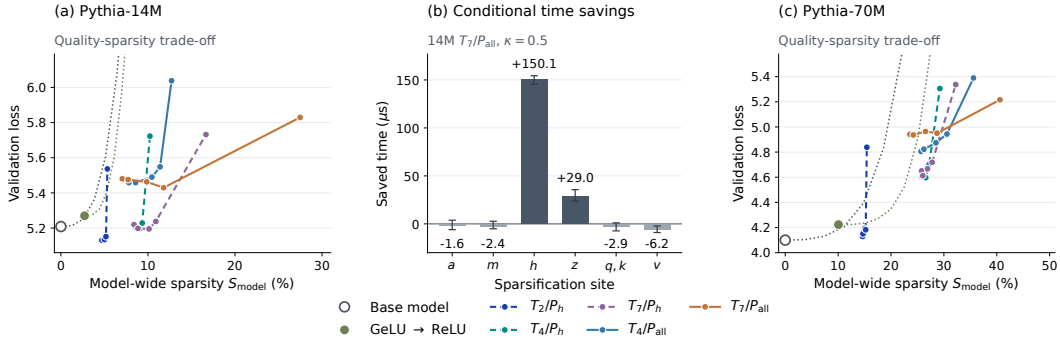


Figure 1: **From broad sparsification to targeted execution and scale transfer.** (a) Pythia-14M validation loss versus model-wide logical sparsity S_{model} exposes the quality costs of different thresholding and pressure placements. (b) The conditional execution ablations in Table 2, measured on the 14M T_7/P_{all} checkpoint at $\kappa = 0.5$, identify h and z as the profitable skipping sites, motivating the narrower T_2/P_h recipe. Saved time is full-model latency with one skipping path disabled minus latency with all paths enabled; whiskers span all differences between the three process means per mode. These effects are not additive. Timings use BF16 on an RTX 5090, batch one and 2,048 tokens; q, k are ablated jointly. (c) Pythia-70M tests the quality–sparsity ordering at a larger scale: T_2/P_h retains near-baseline quality for $\kappa \leq 0.1$, while its most aggressive threshold also incurs a quality cost. Panels (a,c) each show 27 trained checkpoints from the seven shared recipes in the legend; curves connect $\kappa = 0, .01, .05, .1, .5$. Dotted Base/ReLU curves show post-hoc clipping. The two panels use different axis ranges; logical sparsity is not measured speedup.

their individual design choices are difficult to disentangle. We instead factor sparsification into two primitives—activation pressure (P) and thresholding (T)—and study how their placement affects quality, sparsity, and realized execution under matched pretraining conditions.

We first use Pythia-14M (Biderman et al., 2023) as a controlled design-space exploration. We begin with increasingly broad thresholding recipes, T_4 and T_7 , combined with either pressure concentrated at the feed-forward hidden activation (P_h) or pressure distributed across all thresholded sites (P_{all}). We couple these models to a sparse GPU implementation so that the training interventions can be evaluated not only by activation sparsity (S_{model}), but also by realized latency execution. The resulting sweep exposes an important mismatch: although increasing sparsity within a fixed recipe generally reduces latency, aggregate sparsity alone does not determine speed. At aggressive thresholding ($\kappa = 0.5$), T_7/P_{all} is substantially sparser than T_4/P_{all} (27.48% versus 12.71%), yet is slightly slower (0.473 versus 0.459 ms).

This mismatch motivates two diagnostic questions: (i) *where does sparsity actually emerge across the network*, and (ii) *where does that sparsity actually save execution time*? Decomposing sparsity across sites and layers reveals a nonlocal response that we call *sparsity spillover*: pressure applied only at the FFN hidden activation h substantially increases sparsity not only at h , but also at the unpressured attention-output activation z . An independent execution decomposition points to the same locations. Conditional kernel ablations show that the measurable latency savings arise primarily from skipping work at h and z , whereas substantial bypass at other sites does not necessarily overcome the overhead required to exploit it.

This convergence suggests a narrower intervention, T_2/P_h , which applies pressure only at h and thresholds only h and z . At 14M, T_2/P_h preserves quality while exposing sparsity at these two sites. We then promote T_2/P_h together with the broader T_4 and T_7 recipes to Pythia-70M. The quality–sparsity ordering transfers: T_2/P_h remains close to the dense model’s validation loss while approaching the sparsity directly reachable through h and z , while broader recipes attain greater model-wide sparsity at substantially larger quality cost.

Sparse execution, however, does not transfer automatically. A direct adaptation of the 14M kernel to 70M adds enough overhead to offset much of the additional sparse gains. To profitably explore the 70M sparsity, it is required additional kernel development and optimization. These results show that activation sparsification should be viewed as a co-design training-and-execution problem: training

should induce the properties that inference system can exploit, rather than maximizing a generic sparsity metric and attempting to accelerate the resulting representation afterward.

2 RELATED WORK

2.1 SPARSIFICATION INTERVENTIONS

Post-hoc thresholding removes small activations at inference-time of a pre-trained model. While CATS thresholds small-magnitude activations within FFNs (Lee et al., 2024), TEAL extends it to attention (Liu et al., 2024). These studies motivate training-time interventions to adapt activation distributions to sparse computation and improve the trade-off between quality and sparsity.

ReLU Strikes Back studies the effect of activations functions in language models (Mirzadeh et al., 2023), showing clear sparsity gains on the use of ReLUs. ProSparse combines progressive ℓ_1 pressure and thresholding at FFNs during continued training (Song et al., 2024). It does not, however, extend its interventions to attention and restrict the analysis to local FFN sparsity. Cetin et al. (2026) extends ProSparse setup to pre-training and propose specialized kernels. These studies show that training can reshape the quality–sparsity trade-off, but they’re restricted to a single local intervention at FFN sites.

Q-Sparse extend sparsification beyond FFNs. It applies top- k selection to both attention and FFN projection inputs (Wang et al., 2024), but with no ℓ_1 pressure. Its top- k operation introduces, however, additional overhead due to the sorting operation. Spark Transformer tries to address it with statistical top- k (You et al., 2025), but we take a simpler approach and use a constant thresholding non-linearity. We argue that when the intervention is introduced during training, the network can adapt itself. Additionally, we evaluate the complementary effect of regularization pressure on both attention and FFNs.

2.2 INFERENCE SPEEDUP

Turning activation sparsity into speedup requires a specialized kernel adapted to the model architecture. TEAL accelerates batch-one autoregressive decoding, reporting up to $1.8\times$ speedup at 50% sparsity (Liu et al., 2024). Cetin et al. (2026) address batched matrix multiplication with their TwELL kernel, and report $1.3\times$ speedup for batch inference. Spark likewise couples its sparsity mechanism with an implementation that reports decoding gains (You et al., 2025). In each setting, skipped computation and memory access must compensate the additional overhead introduced by the sparse operations. This development has been increasingly accelerated by the use of coding agents, through iterative kernel development with execution and profiling feedback (Ouyang et al., 2025; Luo et al., 2026). We take a related approach and use GPT-6 Astra to develop a specialized kernel based on our training interventions (OpenAI, 2026).

3 METHODOLOGY

3.1 SPARSIFICATION INTERVENTIONS

A sparsification intervention specifies where activations are thresholded (thresholding sites $\mathcal{N}$) and where activation pressure is applied (pressure sites $\mathcal{P}$). The sets $\mathcal{N}$ and $\mathcal{P}$ need not coincide, and the thresholding and pressure functions may take different forms. Together, these choices define a combinatorial space of training interventions. We restrict this space to keep the computational cost manageable while addressing the central research questions.

For thresholding, we avoid the overhead of exact top- k selection (Wang et al., 2024; You et al., 2025) and instead use one-sided and symmetric fixed-threshold operators $G_{+, \kappa}(x) = x\mathbf{1}\{x \geq \kappa\}$ and $G_{\pm, \kappa}(x) = x\mathbf{1}\{|x| \geq \kappa\}$, with $\kappa \geq 0$. The constant κ controls the strength of the intervention and is held fixed within each training condition. At $\kappa = 0$, $G_{+, \kappa}$ reduces to ReLU, whereas $G_{\pm, \kappa}$ reduces to the identity.

For activation pressure, ℓ_1 averages across targeted sites and layers the mean absolute activation at each site after any thresholding. Ordinary L1 pressure minimizes $\mathcal{L}_{\text{task}} + \lambda\ell_1$, where λ controls the

pressure intensity (Song et al., 2024; Cetin et al., 2026). To limit conflict between the ℓ_1 pressure and the task update while reducing the need to tune an additional hyperparameter, we introduce orthogonal ℓ_1 pressure (OL1), inspired by gradient-conflict projection methods (Yu et al., 2020; Hsieh et al., 2024). OL1 adds a separately preconditioned pressure correction after the task-only AdamW update, projects away its component opposing the adaptive task direction, and caps its norm relative to that direction. At a fixed optimizer state, once the norm budget is active, further increasing λ leaves the pressure correction unchanged up to numerical stabilizers. To keep the multisite intervention study tractable, we use a common $\lambda = 1$ and budget $b = 1$ and directly measure whether the budget becomes active during training. Appendix A.1 gives the full formulation, a brief comparison and shows that budget saturation depends strongly on the target set.

Figure 2 locates the intervention sites within the Pythia architecture. The variables a and m denote the normalized inputs to the attention and FFN branches, respectively; h is the FFN hidden activation; z is the concatenated attention context before the output projection; and q , k , and v are the query, key, and value operands. Attention probabilities are denoted by p . We apply $G_{+, \kappa}$ at selected sites in $\{a, m, h, z\}$ and $G_{\pm, \kappa}$ at selected sites in $\{q, k, v\}$. When h is targeted, the thresholding nonlinearity replaces GELU. Table 1 summarizes the studied thresholding and pressure assignments and their sparsity ceilings across model sizes. Section 3.2 defines these ceilings.

Table 1: Intervention recipes and analytic sparsity ceilings. T_1 replaces GeLU with ReLU at h . Pressure uses OL1. For $T_2/T_4/T_7$, we sweep $\kappa \in 0, .01, .05, .1, .5$ with $\lambda = 1$ and $b = 1$; the matched 70M latency sweep for T_2/P_h stops at $\kappa = 0.1$, with its additional $\kappa = 0.5$ endpoint reported separately in Appendix D.1; the 14M T_1/P_1 ablation instead sweeps $\lambda \in .05, .1, .5, 1$ at $b = 1$. Ceilings are $100 \cdot \mathcal{S}_{\text{model}}^{\max}$ at 2,048 tokens. Executed recipes by model size are listed in Appendix C.3.

Recipe	Threshold sites $\mathcal{N}$	Pressure sites $\mathcal{P}$	Ceiling (%)		
			14M	70M	410M
T_0/P_0 (Base Model)	$\emptyset$	$\emptyset$	0.00	0.00	0.00
T_1/P_0	h	$\emptyset$	4.28	12.35	24.93
T_1/P_1	h	h	4.28	12.35	24.93
T_2/P_h	h, z	h	5.35	15.44	31.16
T_4/P_0	a, m, h, z	$\emptyset$	12.83	37.06	74.78
T_4/P_h	a, m, h, z	h	12.83	37.06	74.78
T_4/P_{all}	a, m, h, z	$\mathcal{N}$	12.83	37.06	74.78
T_7/P_0	a, m, h, z, q, k, v	$\emptyset$	29.95	49.42	87.25
T_7/P_h	a, m, h, z, q, k, v	h	29.95	49.42	87.25
T_7/P_{all}	a, m, h, z, q, k, v	$\mathcal{N}$	29.95	49.42	87.25

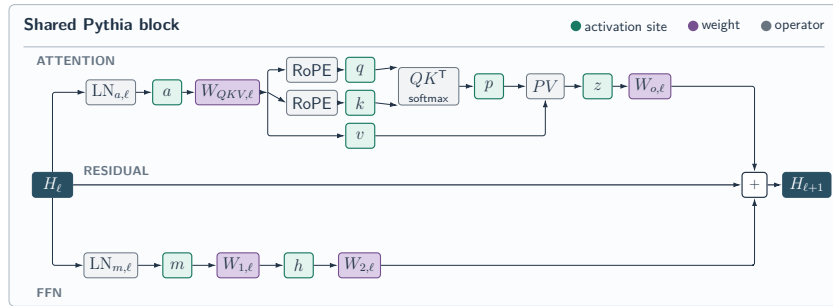


Figure 2: Intervention sites in a shared Pythia block. The attention and FFN branches use separate LayerNorms and join the residual path. Green nodes denote activation sites, and purple nodes denote learned weights. Table 1 specifies the thresholding and pressure assignments.

3.2 SPARSITY MEASURE AND ARCHITECTURAL REACH

Activation sparsity is the fraction of zero activations at a site. We define *model-wide sparsity* as $S_{\text{model}} = N_0/N_{\text{model}}$, where N_0 counts scalar multiplications with a zero activation operand in Pythia-block matrix products, and N_{model} counts all scalar multiplications in those blocks and the final output projection. We aggregate counts over validation data, excluding masked attention pairs.

To quantify intervention potential a priori, we define the *model-wide sparsity ceiling* as $S_{\text{model}}^{\text{max}} = N_{\text{reach}}(\mathcal{N})/N_{\text{model}}$, where $N_{\text{reach}}(\mathcal{N})$ counts scalar multiplications guaranteed a zero activation operand if all selected thresholding sites are 100% sparse. Thus, $S_{\text{model}}^{\text{max}}$ is the maximum sparsity directly attributable to $\mathcal{N}$. It depends on architecture, selected sites, and sequence length. Appendix B gives the precise accounting and definitions.

4 EXPERIMENTAL STUDY

We pretrain Pythia-14M, 70M, and 410M from random initialization (Biderman et al., 2023) on MiniPile (Kaddour, 2023). Each run receives 712 optimizer steps and 1.493 billion input tokens. Within each size, runs share initialization, data order and training budget. We evaluate loss and S_{model} on the full validation set. Appendix C.3 gives the full protocol. For comparison, we apply TEAL-style post-hoc magnitude thresholding (Liu et al., 2024) at a, m, h, z , using percentile targets $p = 0, 0.1, \dots, 0.9$. Site-layer thresholds are calibrated separately on ten training blocks.

Experiments across model sizes. We use 14M for extensive intervention ablations and kernel development, and use 70M as a scale check of whether the observed sparse structures remain exploitable after adapting the implementation. We also evaluated 410M, but it operates in a different optimization regime, exhibiting higher loss than 70M and a larger gradient norm after the same token budget. Because these effects are not isolated, our detailed intervention and runtime analyses focus on 14M and 70M. Appendix D.3 reports the 410M results, and Figure 7 shows the corresponding training trajectories.

Agentic kernel development. We used Codex with GPT-6 Astra (OpenAI, 2026) to develop specialized GPU kernels under human guidance, starting from the implementation of Cetin et al. (2026) and adapting it to Pythia. Candidate changes were tested on a fixed subset of training sequences, and the selected implementation was fixed before evaluation. We then checked full-validation outputs against numerical tolerances and timed a fixed subset of sequences. Appendix E gives the execution paths, correctness checks, and timing protocol. This case illustrates how agentic coding can make execution tests practical for a representation-learning study.

4.1 EXTENSIVE ABLATIONS AT 14M

We first compare the broad T_4 and T_7 recipes at 14M, varying threshold strength and pressure placement (Figure 3). **Stronger interventions extend sparsity at a quality cost.** At $\kappa = 0.5$, T_7/P_{all} reaches 27.48% model-wide sparsity, or 91.8% of its ceiling, while validation loss rises from the base model’s 5.209 to 5.829.

Local pressure preserves quality better than global pressure at moderate thresholds. For $\kappa \leq 0.1$, P_h lowers loss relative to P_{all} in all eight matched T_4/T_7 comparisons, by 0.19–0.32, while sparsity differs by at most 2.1 percentage points. Local pressure also gives lower measured latency in all eight pairs.

More sparsity does not necessarily imply lower latency. Within a fixed recipe, increasing sparsity generally reduces latency, but model-wide sparsity alone is not predictive. At $\kappa = 0.5$, T_7/P_{all} is substantially sparser than T_4/P_{all} (27.48% versus 12.71%), yet slightly slower (0.473 versus 0.459 ms). This mismatch motivates examining where zeros occur and which sites contribute to latency savings.

4.2 SPILLOVER: LOCAL PRESSURE INDUCES NONLOCAL SPARSITY

Local pressure produces a nonlocal response. Figure 4 shows sparsity by site and layer under P_0 , P_h and P_{all} at 14M. At $\kappa = 0.05$, applying pressure only at h increases h sparsity from 74–85% to

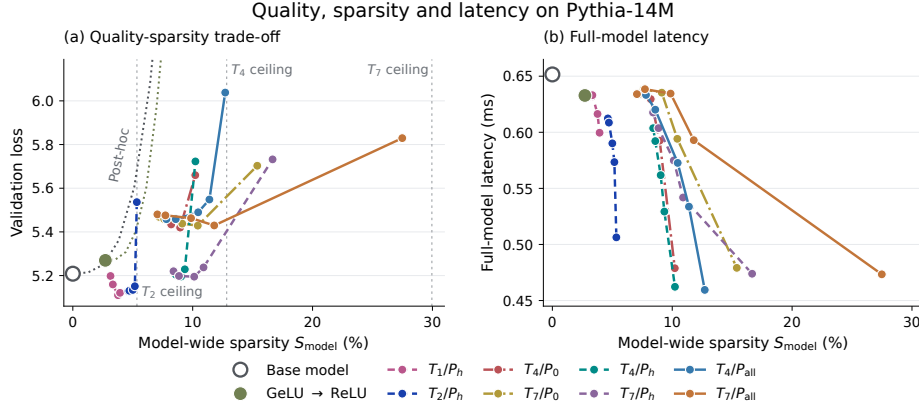


Figure 3: **Quality, sparsity and execution on Pythia-14M.** (a) Final validation loss and (b) full-model latency versus S_{model} for 41 checkpoints spanning the Base/ReLU controls and the T/P intervention grid. Curves connect separately trained settings over $\lambda = .05, .1, .5, 1$ for T_1/P_h and $\kappa = 0, .01, .05, .1, .5$ otherwise. Dotted curves in (a) show post-hoc clipping of the Base/ReLU controls; vertical dotted lines mark analytic sparsity ceilings. Latencies use an RTX 5090, batch one, and 2,048-token sequences.

94–99% for both T_4 and T_7 . The response, however, is not confined to the pressured site: sparsity at the unpressured z activation rises from 71–85% to 83–92% under T_4 , and from 70–87% to 84–94% under T_7 , while changes at a and m are comparatively small.

In the other hand, broadening T_7 pressure to P_{all} makes both h and z less sparse than under P_h in every layer. We speculate this happens because broad pressure reaches the OL1 norm cap much more often: 96.99% of training steps, versus 0.25% under P_h across the threshold grid (Appendix A.1). Competition for this budget may weaken pressure at h . Finally, at $\kappa = 0.5$, h and z already exceed 99.5% sparsity in every layer without pressure, whereas T_7 ’s q, k, v remain at 13–36%. Therefore, some sites may be more susceptible to sparsification than others.

4.3 WHEN SPARSITY TRANSLATES TO SPEEDUP

We next test whether the sites that respond most strongly also save execution time. Table 2 isolates each skipping path at 14M by disabling it while keeping weights, gates and the other paths fixed. It reports *bypass* (the percentage of omitted matrix instructions) and conditional saved time for T_7/P_{all} at $\kappa = 0.5$. **The measurable savings come from h and z :** 150.1 μs and 29.0 μs , respectively. The a, m and joint q, k effects remain unresolved against process variation, whereas PV skipping adds 6.2 μs . These effects are conditional on the other paths remaining enabled and are not additive allocations of total savings.

Sparsity reduces latency when avoided work exceeds the overhead of detecting and skipping it. At h, z , the sparse kernel checks 8×16 activation tiles before loading their associated weights, so empty tiles avoid both weight reads and matrix multiplication. Applying the same strategy at a and m instead increased full-model latency by 17.2% and 27.4% on this checkpoint, with unchanged outputs. Only 11.12% of a ’s 8×16 tiles were empty, and none at m , so most groups still required matrix multiplication and the additional overhead was not justified.

Attention follows FlashAttention’s tiled computation (Dao et al., 2022), but our zero checks occur after loading the operands. Skipping a matrix instruction therefore avoids arithmetic without avoiding those reads or the masking and softmax work. Softmax also turns zero scores into nonzero probabilities, so q, k sparsity does not automatically propagate to PV . Enabling QK/PV skipping together raises full-model latency from 0.514 to 0.521 ms despite substantial bypass. Thus, for this checkpoint and implementation, attention’s arithmetic savings do not offset its overhead, while h/z benefit from avoiding weight reads as well as computation.

The sparsity and execution diagnostics converge on h, z . This motivates T_2/P_h : pressure remains local to h , while thresholding is restricted to h, z . At 14M and $\kappa = 0.1$, it reaches 5.17% model-

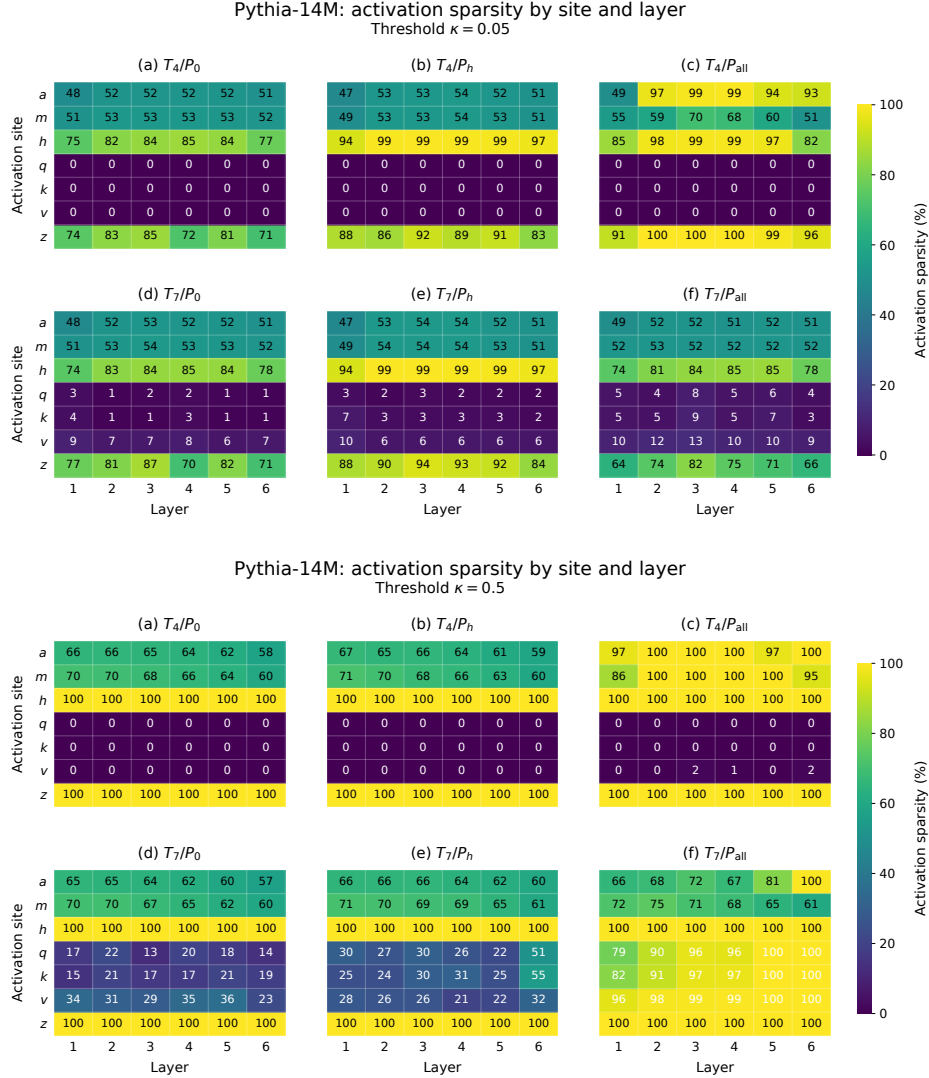


Figure 4: **Activation sparsity by site and layer in Pythia-14M.** The six threshold/pressure recipes at $\kappa = 0.05$ (top) and 0.5 (bottom). Each cell gives the percentage of zero activations over all complete validation sequences at one site and layer. Q/K are post-RoPE; all panels share a 0–100% color scale. Values are rounded to whole percentages, so 0 or 100 does not establish a completely dense or all-zero tensor.

Table 2: **Conditional latency savings:** 14M T_7/P_{all} , $\kappa = 0.5$. Latency is measured per 2,048-token sequence on RTX 5090 (BF16, batch one). Saved time is full-model latency with one skipping path off minus latency with all paths on; positive values indicate a benefit (reduced latency). Brackets give [minimum, maximum] savings across all off/on pairs of three independent timing runs. Each run reports the geometric mean over 64 sequences.

Operation	Bypass (%)	Saved time (μ s)	Process span (μ s)
a : QKV projection	8.4	−1.6	[−6.0, +3.8]
m : FFN-up	0.0	−2.4	[−5.1, +2.7]
h : FFN-down	94.5	+150.1	[+145.4, +154.4]
z : attention-output projection	99.6	+29.0	[+23.7, +35.7]
q, k : QK scores	58.0	−2.9	[−7.3, +1.1]
v : PV product	67.2	−6.2	[−9.0, −2.2]

wide sparsity against a 5.35% ceiling, with loss 5.151 versus Base’s 5.209, and latency 0.573 ms versus native Base’s 0.655 ms (Figure 3; Appendix E). The narrower recipe preserves quality while concentrating sparsity at the sites identified as profitable in the execution analysis.

4.4 RESULTS AT 70M

We promote T_2/P_h together with T_4 and T_7 under local or global pressure to 70M (Figure 5). T_2/P_h **preserves quality best among these three threshold topologies**, with the lowest validation loss at every matched κ . For $\kappa \leq 0.1$, its loss is 4.129–4.193 versus Base’s 4.100, at 14.63–15.21% sparsity against a 15.44% ceiling. Broader interventions extend sparsity further at greater quality cost: T_7/P_{all} reaches 40.60% at $\kappa = 0.5$, or 82.2% of its ceiling, with loss 5.216 (1.12 above Base).

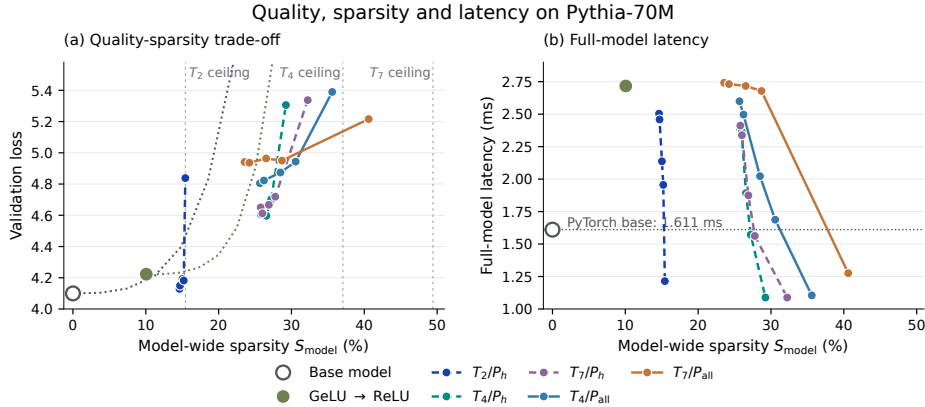


Figure 5: **Quality, sparsity and execution on Pythia-70M.** (a) Validation loss and (b) full-model latency versus S_{model} for 27 checkpoints: Base, GeLU $\rightarrow$ ReLU, T_2/P_h , and T_4/T_7 with P_h or P_{all} , each thresholded recipe at $\kappa = 0, .01, .05, .1, .5$. Colors and line styles match Figure 1. Dotted curves in (a) apply post-hoc clipping to the fixed Base/ReLU controls; high-loss tails extend beyond the view. Vertical guides mark sparsity ceilings. In (b), the Base marker and horizontal reference use native PyTorch/SDPA (1.611 ms); all other markers use the optimized 70M kernel. Measurements use RTX 5090 GPUs, three fresh processes per checkpoint, batch one, 2,048 tokens and full output logits. Appendix E compares the native, original-port and optimized implementations and reports the timing references.

The 14M kernel does not transfer efficiently to 70M. The original port takes 3.325 ms on the base model, compared with 1.611 ms for native PyTorch, and its best sparse recipe reaches only $1.013\times$ the native base-model speed. Further optimization improves sparse h, z execution and dense components, reducing T_7/P_h latency at $\kappa = 0.5$ from 1.590 to 1.088 ms. The resulting $1.481\times$ speedup over native Base is an **end-to-end comparison**: it includes both a different checkpoint and sparse and dense implementation changes. All 27 checkpoints pass full-validation numerical checks in three fresh processes (Appendix E).

The specialized h, z path provides an additional execution benefit at 70M. In a separate controlled session, the optimized T_7/P_h checkpoint at $\kappa = 0.5$ takes 1.128 ms. Replacing only its specialized h, z implementation with native dense operations, while retaining the same weights, thresholds and other optimized components, raises latency to 1.365 ms. The specialized path therefore reduces full-model latency by 17.4% relative to this control. This conditional benefit includes fusion, layout and inspection costs alongside sparsity exploitation (Appendix E).

The lower-loss T_2/P_h points at $\kappa \leq 0.1$, however, still take 1.956–2.504 ms after optimization, slower than the 1.611 ms native Base. At $\kappa = 0.5$, latency falls to 1.214 ms ($1.364\times$ its same-session native Base at 1.656 ms), but loss rises to 4.838. The quality ordering therefore transfers to 70M, while the favorable 14M quality–latency balance does not transfer directly.

5 DISCUSSION AND CONCLUSION

Our results suggest that sparsification should be understood as a joint training-and-deployment problem rather than as a property of activation distributions alone. First, **local pressure does not imply a local response**: applying pressure only at the FFN hidden site can alter sparsity at other sites, showing that the network can reorganize its internal representations in response to a localized training intervention. More broadly, **sparsity is useful only if the resulting zero structure can be exploited by the inference system**. S_{model} is best viewed as a training-time proxy for computational opportunity, not as a direct predictor of latency. In the controlled 14M measurements, the largest latency gains come from (h, z) sites, which were the ones more susceptible to sparsification, with higher achieved sparsity. At 70M, the 14M implementation required further optimization; the specialized h, z path then reduces latency by 17.4% relative to native h, z operations on the same checkpoint, while dense-kernel improvements also contribute to the end-to-end gains (Appendix E). One hypothesis is that we must achieve a high ($> 90\%$) level of site-layer sparsity, so the zeros gets a exploitable tiled structure with higher probability.

Transformers appear sufficiently flexible to adapt their internal representations to architectural and optimization constraints, suggesting that larger gains may come from **designing training objectives around inference requirements**, rather than first maximizing a generic sparsity metric and only afterward attempting to accelerate it. The same principle may extend beyond activation sparsity: training could be used to shape representations for properties required by weight sparsity, quantization, batched execution, or other hardware-specific constraints. In this view, deployment requirements are not merely constraints imposed after training, but potential signals for deciding what properties the network should learn.

Several limitations define the scope of these conclusions and point to concrete directions for further work. Our scaling evidence is limited: although the 410M experiments demonstrate that high model-wide sparsity remains attainable, the fixed token budget leaves 410M in a substantially different optimization regime, so it should be interpreted as a stress test rather than a clean scaling experiment. Our runtime study is also restricted to batch-one, full-sequence inference on an RTX5090; we do not study how sparsity interacts with batched inference, cached decoding, or other hardware.

In addition, the full intervention space is combinatorial, and under the available compute budget we evaluate only a selected set of threshold and pressure placements rather than an exhaustive factorial study. The nonlocal response visible in Figure 4 motivates interventions concentrated at h and z . The T_2/P_h results at 14M support this direction: they preserve much of the execution benefit while avoiding thresholding at sites whose sparsity is expensive in quality or unprofitable for the current kernel. Whether this balance transfers depends on the implementation as well as the sparse structure. Related extensions include learning or tuning different thresholds κ at different sites and directly sparsifying attention probabilities after softmax, which could expose work in pv without relying on q/k zeros to propagate through softmax. More generally, rather than asking where the model can be made sparse, a productive systems-guided approach is to decompose full-model inference into individual operations, identify where latency and removable work are concentrated, determine what structured representation would allow those operations to be skipped efficiently, and then design training-time interventions that encourage precisely that structure.

AI USE STATEMENT

We used OpenAI Codex with GPT-6 Astra to assist with experimental design refinement, mathematical exposition, implementation and debugging of training and evaluation code, GPU-kernel development, experiment execution and monitoring, analysis and interpretation of results, figure preparation, and manuscript drafting and revision. The authors directed the research and approved experimental designs. AI-assisted implementations were checked through tests and comparisons with reference computations; the kernel validation procedures and their limitations are documented in Appendix E. The authors take responsibility for the final text, code, results and scientific claims.

REFERENCES

- Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 2397–2430. PMLR, 2023. URL <https://proceedings.mlr.press/v202/biderman23a.html>.
- Edoardo Cetin, Stefano Peluchetti, Emilio Castillo, Akira Naruse, Mana Murakami, and Llion Jones. Sparser, Faster, Lighter Transformer Language Models. *arXiv preprint arXiv:2603.23198*, 2026. URL <https://arxiv.org/abs/2603.23198>.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *arXiv preprint arXiv:2205.14135*, 2022. URL <https://arxiv.org/abs/2205.14135>.
- Yu-Guan Hsieh, James Thornton, Eugene Ndiaye, Michal Klein, Marco Cuturi, and Pierre Ablin. Careful with that Scalpel: Improving Gradient Surgery with an EMA. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 19085–19100. PMLR, 2024. URL <https://proceedings.mlr.press/v235/hsieh24a.html>.
- Jean Kaddour. The MiniPile Challenge for Data-Efficient Language Models. *arXiv preprint arXiv:2304.08442*, 2023. URL <https://arxiv.org/abs/2304.08442>.
- Donghyun Lee, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models. In *Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=v3w2a7EInO>.
- James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-Free Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2408.14690*, 2024. URL <https://arxiv.org/abs/2408.14690>.
- Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Mao Luo, Hongbin Li, Feng Lin, Hanling Yi, and Zhe Huang. Agentic Kernel Optimization: Generating State-of-the-Art GPU Kernels Without Hand-Written CUDA. *arXiv preprint arXiv:2608.14560*, 2026. URL <https://arxiv.org/abs/2608.14560>.
- Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C. Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2310.04564*, 2023. URL <https://arxiv.org/abs/2310.04564>.
- OpenAI. GPT-6 Astra: A New Generation of Intelligence. OpenAI, September 2026. URL <https://openai.com/index/gpt-6-astra/>. Accessed September 19, 2026.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs Write Efficient GPU Kernels? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 47356–47415. PMLR, 2025. URL <https://proceedings.mlr.press/v267/ouyang25a.html>.
- Chenyang Song, Xu Han, Zhengyan Zhang, Shengding Hu, Xiyu Shi, Kuai Li, Chen Chen, Zhiyuan Liu, Guangli Li, Tao Yang, and Maosong Sun. ProSparse: Introducing and Enhancing Intrinsic Activation Sparsity within Large Language Models. *arXiv preprint arXiv:2402.13516*, 2024. URL <https://arxiv.org/abs/2402.13516>.

Hongyu Wang, Shuming Ma, Ruiping Wang, and Furu Wei. Q-Sparse: All Large Language Models can be Fully Sparsely-Activated. *arXiv preprint arXiv:2407.10969*, 2024. URL <https://arxiv.org/abs/2407.10969>.

Chong You, Kan Wu, Zhipeng Jia, Lin Chen, Srinadh Bhojanapalli, Jiaxian Guo, Utku Evci, Jan Wassenberg, Praneeth Netrapalli, Jeremiah J. Willcock, Suvinay Subramanian, Felix Chern, Alek Andreev, Shreya Pathak, Felix Yu, Prateek Jain, David E. Culler, Henry M. Levy, and Sanjiv Kumar. Spark Transformer: Reactivating Sparsity in FFN and Attention. *arXiv preprint arXiv:2506.06644*, 2025. URL <https://arxiv.org/abs/2506.06644>.

Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient Surgery for Multi-Task Learning. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2001.06782>.

A INTERVENTION DEFINITIONS

A.1 ACTIVATION PRESSURE AND OL1

Objective and update. For J activation tensors A_j at the sites and layers targeted by $\mathcal{P}$, each with n_j entries after any thresholding, define

$$\ell_1(\theta; \mathcal{P}) = \frac{1}{J} \sum_{j=1}^J \frac{1}{n_j} \sum_{i=1}^{n_j} |A_{j,i}|. \quad (1)$$

Each tensor has equal weight, so changing $\mathcal{P}$ changes both the targets and normalization. Ordinary L1 applies AdamW (Loshchilov & Hutter, 2019) to the globally clipped gradient of $\mathcal{L}_{\text{task}} + \lambda \ell_1$. OL1 separates task and pressure updates: task-only AdamW is followed by a pressure correction that removes the opposing component and limits its norm. Pressure therefore does not directly enter Adam’s moment estimates.

Both gradients are evaluated at the same parameters and averaged over the same microbatches. The clipped task gradient updates AdamW’s bias-corrected moments $\hat{m}_t, \hat{v}_t$; the pressure gradient $g_1 = \nabla_{\theta} \ell_1$ is unclipped and has no momentum. Using the same preconditioner for both directions gives

$$u = \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon_A}, \quad w = \frac{g_1}{\sqrt{\hat{v}_t} + \epsilon_A}. \quad (2)$$

Norms and inner products include parameters with both gradients and initialized Adam state. For $c = \langle u, w \rangle$, $q = \|u\|_2^2$ and stabilizer $\epsilon > 0$,

$$\tilde{w} = \begin{cases} w - \frac{c}{q + \epsilon} u, & c < 0 \text{ and } q > \epsilon, \\ w, & \text{otherwise.} \end{cases} \quad (3)$$

Aligned pressure is retained. For weight $\lambda \geq 0$ and relative norm budget $b > 0$,

$$r = \frac{\lambda \|\tilde{w}\|_2}{\|u\|_2 + \epsilon}, \quad s = \min \left\{ 1, \frac{b}{r + \epsilon} \right\} \quad (r > 0), \quad (4)$$

with $s = 1$ when $r = 0$. After task-only AdamW, each parameter group p receives $\theta_p \leftarrow \theta_p - \alpha_p \lambda s \tilde{w}_p$, where α_p is its learning rate. Before learning-rate scaling and ignoring stabilizers, the correction norm is $\min\{\lambda \|\tilde{w}\|_2, b \|u\|_2\}$. At a fixed optimizer state, increasing λ has no further effect once the cap is active.

OL1 combines conflict projection (Yu et al., 2020) and a smoothed task direction (Hsieh et al., 2024) in Adam’s preconditioned coordinates. The stabilizer makes orthogonality approximate. This calculation excludes group learning rates and decoupled weight decay; avoiding opposition to the Adam direction does not guarantee unchanged task or validation loss.

L1 versus OL1. At 14M T_1/P_1 , matched initialization, data order and training budget yield similar results (Table 3). Sparsity differs by less than 0.02 percentage points at every λ . OL1 has slightly lower loss at $\lambda \in \{0.05, 0.1, 0.5\}$ and higher loss at $\lambda = 1$. One seed cannot establish whether these small loss differences persist across seeds.

Table 3: **L1 and OL1 at T_1/P_1 on Pythia-14M.** Final validation loss and model-wide sparsity after 712 matched training steps; OL1 uses $b = 1$. Both metrics cover all 338 complete validation blocks (500 documents; 1,444 tail tokens excluded). One initialization per condition.

λ	L1		OL1	
	S_{model} (%)	Val. loss	S_{model} (%)	Val. loss
0.05	3.1416	5.2061	3.1452	5.1981
0.1	3.3363	5.1655	3.3386	5.1594
0.5	3.7876	5.1127	3.7680	5.1102
1	3.9493	5.1023	3.9384	5.1210

Update geometry and budget saturation. We measure $\rho_{\text{opp}} = \max(0, -c)/(q + \epsilon)$, the removed component’s norm relative to $\|u\|_2$ before capping. All recorded steps satisfy $q > \epsilon$; aligned steps contribute zero. Figure 6 shows ρ_{opp} and r/b for the 20 14M OL1 conditions in Figure 1, using all 712 updates per condition at $\lambda = b = 1$. The cap is active ($s < 1$) on 96.99% of T_7/P_{all} steps, compared with 0.51% for T_4/P_{all} and 0.25% for either P_h recipe. The diagnostics are consistent with stronger pressure–task conflict when attention sites are included, but do not isolate each site’s contribution.

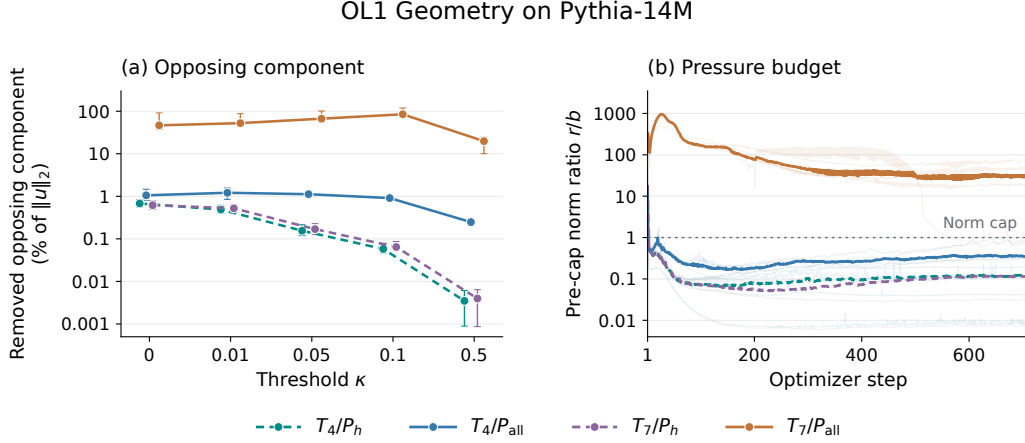


Figure 6: **OL1 geometry depends on pressure placement at 14M.** (a) Median removed opposing component, $100\rho_{\text{opp}}$; bars span the 25th–75th percentiles over 712 training steps per condition, not confidence intervals. (b) Pre-cap norm ratio r/b : faint curves show individual thresholds, and bold curves their pointwise median. The dotted line marks the norm cap. Colors and line styles match Figure 1; both y-axes are logarithmic, and $\lambda = b = 1$ throughout.

B SPARSITY ACCOUNTING

B.1 ACTIVATION AND MODEL-WIDE SPARSITY

Activation sparsity is the fraction of zero entries at a site, pooling counts across layers and validation sequences before dividing.

Model-wide sparsity counts scalar multiplications with a zero activation input:

$$\mathcal{S}_{\text{model}} = \frac{N_0}{N_{\text{model}}}. \quad (5)$$

N_0 counts these multiplications within Pythia blocks; N_{model} counts all multiplications in those blocks and the final vocabulary projection. The latter contributes only to the denominator. Counts are pooled across layers and validation sequences; percentages are $100\mathcal{S}_{\text{model}}$.

For a block’s linear projection from d_{in} to d_{out} features over n rows, the denominator contribution is $nd_{\text{in}}d_{\text{out}}$. Each zero input contributes d_{out} to N_0 , one multiplication per output feature. Zero weights do not count as activation sparsity.

In attention, both operands are activations. We count each multiplication once if either or both are zero. For queries q , keys k , values v and attention probabilities p , one head in one sequence contributes

$$N_{0,\text{QK}} = \sum_{t=1}^T \sum_{u=1}^t \sum_{j=1}^{d_h} \mathbf{1}\{q_{tj} = 0 \vee k_{uj} = 0\}, \quad (6)$$

$$N_{0,\text{PV}} = \sum_{t=1}^T \sum_{u=1}^t \sum_{j=1}^{d_h} \mathbf{1}\{p_{tu} = 0 \vee v_{uj} = 0\}. \quad (7)$$

Here T is sequence length, d_h the head dimension, t the query position and u the key/value position. Restricting $u \leq t$ excludes masked future positions from both counts. We sum over heads, layers and validation sequences, using the actual operands after thresholding and intervening transformations.

Normalization, softmax, activation functions, additions and memory transfers are excluded, so $\mathcal{S}_{\text{model}}$ does not directly measure latency or speedup.

B.2 MODEL-WIDE SPARSITY CEILING

Assume all activations at the thresholding sites $\mathcal{N}$ are zero. If $N_{\text{reach}}(\mathcal{N})$ counts the multiplications that must then have a zero activation input, the ceiling is

$$\mathcal{S}_{\text{model}}^{\text{max}} = \frac{N_{\text{reach}}(\mathcal{N})}{N_{\text{model}}}. \quad (8)$$

The denominator is unchanged, and each multiplication is counted once. The numerator includes downstream activations forced to zero: setting $v = 0$ also makes $z = 0$, affecting both PV and the attention-output projection. Zero attention scores do not imply zero softmax probabilities.

The ceiling depends only on architecture, selected sites and sequence length. Natural zeros outside those sites can raise measured $\mathcal{S}_{\text{model}}$ above it; the ceiling does not predict the associated quality cost. Appendix C.2 gives the counts for each Pythia size.

C EXPERIMENTAL SETUP DETAILS

C.1 PYTHIA ARCHITECTURE AND INTERVENTION SITES

Table 4 gives the shapes and positions of the sites in Figure 2. Here T is sequence length, d hidden width, d_f FFN width and H the number of heads, each with dimension $d_h = d/H$. Sites and threshold are shared across layers.

Table 4: Activation shapes and intervention positions for one sequence.

Site	Shape	Following matrix product	Position
a	$T \times d$	QKV projection	After attention LayerNorm
m	$T \times d$	FFN up projection	After FFN LayerNorm
h	$T \times d_f$	FFN down projection	Activation-function output
q, k	$H \times T \times d_h$	Attention scores	After RoPE
v	$H \times T \times d_h$	Probability-value	After the QKV split
z	$T \times d$	Attention-output projection	Concatenated attention context

C.2 ARCHITECTURE-SPECIFIC SPARSITY CEILINGS

For one sequence, each Pythia block uses $3Td^2$ scalar multiplications in the QKV projection, Td^2 in the attention-output projection, Tdd_f in each FFN projection, and $dT(T+1)/2$ in each of QK and PV . With L layers and vocabulary size V_{vocab} , the totals are

$$\begin{aligned} C_{\text{block}} &= T(4d^2 + 2dd_f) + dT(T+1), \\ C_{\text{model}} &= LC_{\text{block}} + TdV_{\text{vocab}}. \end{aligned} \quad (9)$$

Sequence count cancels from the ceiling ratio. For $d_f = 4d$, per-layer counts in the ceiling numerator are 0 for T_0/P_0 , $4Td^2$ for T_1/P_0 and T_1/P_1 , $12Td^2$ for T_4 , and C_{block} for T_7 . Multiplying by L/C_{model} gives $\mathcal{S}_{\text{model}}^{\text{max}}$, independent of pressure placement.

Table 1 uses $V_{\text{vocab}} = 50,304$ and $(L, d) = (6, 128), (6, 512), (24, 1,024)$ for 14M, 70M and 410M, respectively, with the common sequence length in Table 5.¹

¹Pinned architecture configurations: 14M, 70M, and 410M.

C.3 TRAINING AND EVALUATION

All sizes use the Pythia-14M-deduped tokenizer and seeds of 1234 for initialization and data order, with one seed per condition. AdamW settings are shared: $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, weight decay 0.1 and gradient clipping at norm 1. Attention and hidden dropout are zero. Training uses 712 updates with 1% linear warmup followed by cosine learning-rate decay. Table 5 gives the learning rates, microbatch sizes and shared effective batch. Gradients accumulate over the listed microbatches before each optimizer update.

Table 5: **Training settings across model sizes.** Effective batch size equals microbatch size times gradient accumulation. The common training budget gives fewer tokens per parameter at larger sizes.

Setting	14M	70M	410M
Parameters	14,067,712	70,426,624	405,334,016
Peak learning rate	10^{-3}	10^{-3}	3×10^{-4}
Minimum learning rate	10^{-4}	10^{-4}	3×10^{-5}
Microbatch (sequences)	32	4	4
Gradient accumulation (microbatches)	32	256	256
Effective batch (sequences/update)	1,024	1,024	1,024
Sequence length	2,048	2,048	2,048
Tokens per parameter	106.14	21.20	3.68

All sizes include the T_0/P_0 and T_1/P_0 controls. The 14M study adds T_1/P_1 with L1 or OL1, and T_4 and T_7 with P_0 , P_h or P_{all} . Both 14M and 70M also include T_2/P_h ; its 70M sweep uses $\kappa = 0, .01, .05, .1$ in the matched latency sweep, with the additional $\kappa = 0.5$ endpoint reported separately in Appendix D.1. At 70M, T_4 and T_7 use P_h or P_{all} ; at 410M, only P_{all} . Parameter grids are given in Table 1. We evaluate the final checkpoint, without validation-based selection. The main figures measure loss and model-wide sparsity on the same validation sequences (Appendix B).

Figure 7 shows the training trajectories. They do not establish convergence, and the lower learning rates at 410M limit comparisons that attribute loss differences to model size alone.

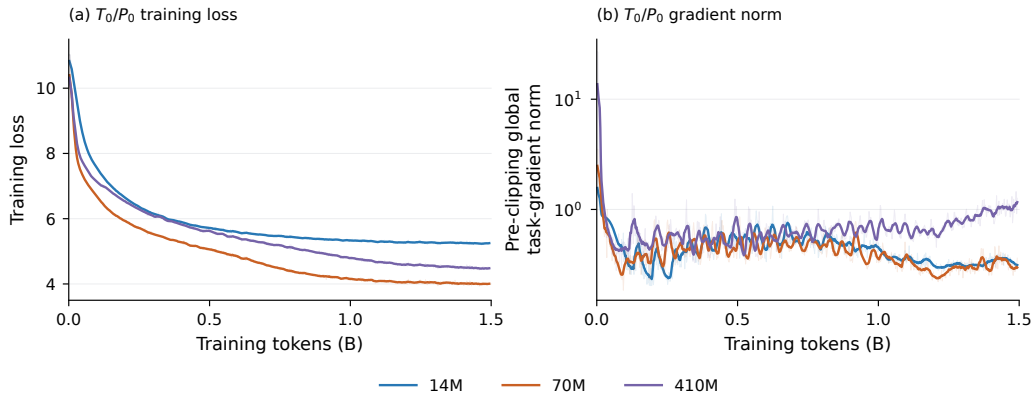


Figure 7: **Base-model (T_0/P_0) training across model sizes.** (a) Training loss and (b) gradient norm before clipping (logarithmic scale). Faint lines show individual updates; solid lines show centered nine-update means, with shorter windows at the ends. Gradient norms are computed after accumulation and are not normalized by parameter count.

D COMPLETE RESULTS AND SUPPORTING DIAGNOSTICS

D.1 DETAILED TRAINED RESULTS

Tables 6–9 report all 84 trained conditions, including settings outside the favorable quality–sparsity trade-off. Loss and model-wide sparsity use the same definitions and final checkpoints as the main figures, with one training seed and complete validation per condition. Values are rounded.

Table 6: **Additional 14M ablations.** Pressure at h (top) and thresholding without pressure (bottom). Controls and T_2/P_h appear in Table 8.

λ	T_1/P_1 (L1)		T_1/P_1 (OL1)	
	Loss	$\mathcal{S}_{\text{model}}$ (%)	Loss	$\mathcal{S}_{\text{model}}$ (%)
0.05	5.2061	3.142	5.1981	3.145
0.1	5.1655	3.336	5.1594	3.339
0.5	5.1127	3.788	5.1102	3.768
1	5.1023	3.949	5.1210	3.938
κ	T_4/P_0		T_7/P_0	
	Loss	$\mathcal{S}_{\text{model}}$ (%)	Loss	$\mathcal{S}_{\text{model}}$ (%)
0	5.4705	7.212	5.4684	7.218
0.01	5.4665	7.414	5.4588	7.618
0.05	5.4341	8.206	5.4379	9.127
0.1	5.4196	8.953	5.4287	10.425
0.5	5.6597	10.216	5.7029	15.387

The additional 70M T_2/P_h checkpoint at $\kappa = 0.5$ was measured in a later session with the same frozen optimized kernel. It reaches 15.427% sparsity at loss 4.8380, with latency reduced from 1.696 ms for the original port to 1.214 ms. Table 7 reports Base and $\kappa = 0.1$ repeated in that session: optimized latency changes by 2.19% and 0.20%, respectively, while native Base changes by 2.80%. Figure 5 retains the original 26 timings and marks this later endpoint separately, without pooling or rescaling.

Table 7: **Later 70M measurement and session references.** The frozen optimized kernel is evaluated on the additional T_2/P_h , $\kappa = 0.5$ checkpoint, with Base and $\kappa = 0.1$ repeated in the same later RTX 5090 session. Each row uses three fresh processes, 1,344 timings per implementation and full-validation numerical checks. Earlier rows are retained for comparison; sessions are not pooled.

Checkpoint	Session	Native (ms)	Port (ms)	Optimized (ms)
Base	Earlier	1.611	3.325	2.732
	Later	1.656	3.314	2.792
T_2/P_h , $\kappa = 0.1$	Earlier	1.662	2.258	1.956
	Later	1.721	2.259	1.959
T_2/P_h , $\kappa = 0.5$	Later	1.720	1.696	1.214

Table 8: **14M/70M results.** OL1 pressure uses $\lambda = b = 1$. The original 26 70M checkpoints share one RTX 5090 session (native Base: 1.611 ms). The T_2/P_h , $\kappa = 0.5$ row uses a later session with the same frozen kernel and protocol; its repeated references are given in Table 7. Absolute timings are not pooled or rescaled across sessions.

Recipe	κ	14M		70M		70M latency (ms)	
		Loss	$\mathcal{S}_{\text{model}}$ (%)	Loss	$\mathcal{S}_{\text{model}}$ (%)	Port	Optimized
T_0/P_0 (Base)	—	5.2086	0.000	4.0998	0.000	3.325	2.732
T_1/P_0 (ReLU)	—	5.2696	2.714	4.2227	10.066	3.292	2.717
T_2/P_h	0	5.1286	4.631	4.1287	14.630	2.730	2.504
	0.01	5.1306	4.717	4.1499	14.699	2.684	2.459
	0.05	5.1344	5.004	4.1925	15.030	2.409	2.137
	0.1	5.1511	5.175	4.1823	15.212	2.258	1.956
	0.5	5.5363	5.339	4.8380	15.427	1.696	1.214
T_4/P_h	0	5.2157	8.409	4.6065	25.792	2.597	2.383
	0.01	5.2045	8.586	4.6185	25.971	2.555	2.365
	0.05	5.1956	9.036	4.5964	26.566	2.229	1.892
	0.1	5.2287	9.343	4.7020	27.218	1.954	1.573
	0.5	5.7227	10.227	5.3057	29.236	1.593	1.088
T_4/P_{all}	0	5.4582	7.810	4.8053	25.673	2.774	2.599
	0.01	5.4583	8.587	4.8227	26.224	2.689	2.497
	0.05	5.4898	10.456	4.8739	28.490	2.286	2.023
	0.1	5.5483	11.394	4.9437	30.574	2.042	1.687
	0.5	6.0380	12.713	5.3895	35.596	1.598	1.105
T_7/P_h	0	5.2198	8.399	4.6508	25.786	2.624	2.412
	0.01	5.1981	8.856	4.6122	26.009	2.540	2.338
	0.05	5.1950	10.126	4.6670	26.895	2.217	1.875
	0.1	5.2374	10.904	4.7189	27.779	1.944	1.562
	0.5	5.7320	16.664	5.3372	32.233	1.590	1.088
T_7/P_{all}	0	5.4802	7.054	4.9412	23.562	3.143	2.741
	0.01	5.4758	7.723	4.9364	24.226	3.098	2.732
	0.05	5.4628	9.863	4.9636	26.524	2.960	2.717
	0.1	5.4295	11.797	4.9501	28.694	2.869	2.678
	0.5	5.8294	27.483	5.2160	40.602	1.718	1.276

D.2 POST-HOC CLIPPING AT 14M

Figure 8 extends the base/ReLU comparison in Figure 1 to the trained checkpoints with retained post-hoc sweeps. The detailed view resolves the trade-offs near the trained models; the full-range view shows the loss cost of aggressive clipping. These are changes to fixed models at evaluation, rather than additional training conditions. The clipping protocol is the same as in Section 4. No corresponding sweep was collected for T_2/P_h , T_4/P_h or T_7/P_h .

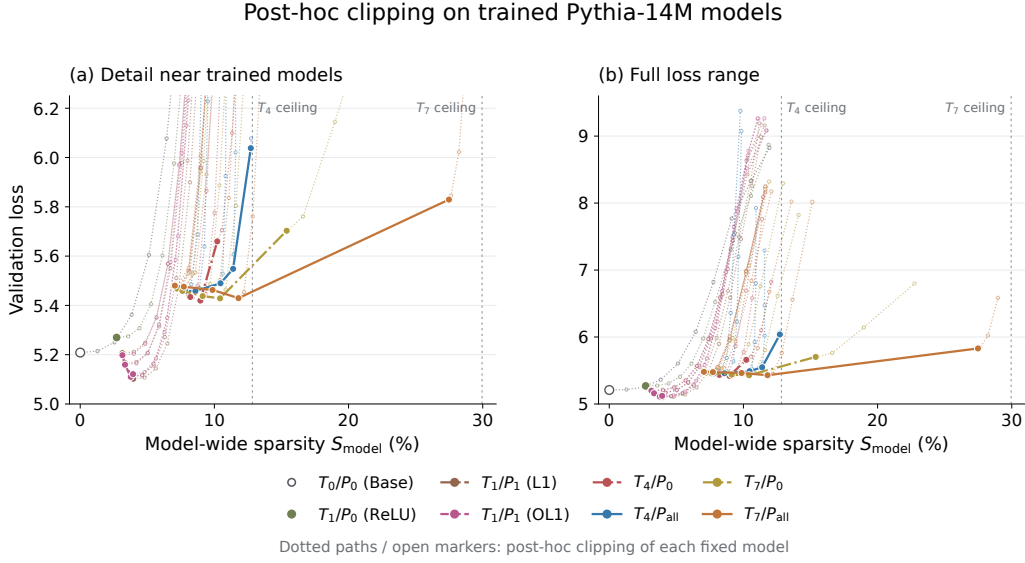


Figure 8: **Post-hoc clipping on trained Pythia-14M models.** Thirty trained checkpoints and ten clipping targets per checkpoint ($p = 0, 0.1, \dots, 0.9$). Both panels show the same settings, with different loss ranges. Prominent markers and recipe lines identify trained models; dotted paths with small open markers follow each fixed model through increasing clipping targets. Colors and recipe labels match the main figures, with the L1 comparison included explicitly. Each path starts at its measured $p = 0$ point. Lines connect evaluated settings, not fitted frontiers.

D.3 410M RESULTS

410M as a fixed-token stress test. Figure 9 retains all twelve trained 410M conditions and both complete control-clipping paths. At $\kappa = 0.5$, T_7/P_{all} reaches 80.62% model-wide sparsity, or 92.4% of the seven-site ceiling, at loss 5.121 versus the base model’s 4.547. This tests attainable sparsity under a larger architecture and the same token budget; it is not a quality-scaling experiment. The 410M seven-site ceiling is 87.25%, compared with 49.42% at 70M, so greater architectural reach also contributes to the larger observed model-wide sparsity.

The 410M base-model loss exceeds 70M’s 4.100. All sizes receive 712 optimizer steps and 1.493B input tokens, corresponding to 106.14, 21.20 and 3.68 tokens per parameter at 14M, 70M and 410M. The 410M schedule also has a lower peak learning rate (3×10^{-4} rather than 10^{-3}). These differences may contribute to the non-monotonic quality ordering, but their effects are not isolated. The training trajectories are retained in Figure 7. We therefore treat 410M as a limited stress test and use 14M/70M for detailed intervention and runtime analysis.

Table 9: **410M fixed-token stress test.** All twelve trained conditions; pressure uses OL1 with $\lambda = b = 1$. This size has no P_h or unpressured T_4/T_7 runs.

Recipe	κ	Loss	$\mathcal{S}_{\text{model}}$ (%)
T_0/P_0 (Base)	—	4.5475	0.011
T_1/P_0 (ReLU)	—	4.6513	21.050
T_4/P_{all}	0	5.6921	38.339
	0.01	5.6914	39.149
	0.05	5.6789	42.125
	0.1	5.6789	45.688
	0.5	5.1911	71.591
T_7/P_{all}	0	5.4263	41.121
	0.01	5.4331	41.478
	0.05	5.4679	44.053
	0.1	5.4908	48.998
	0.5	5.1209	80.616

Pythia-410M: fixed-token sparsity stress test

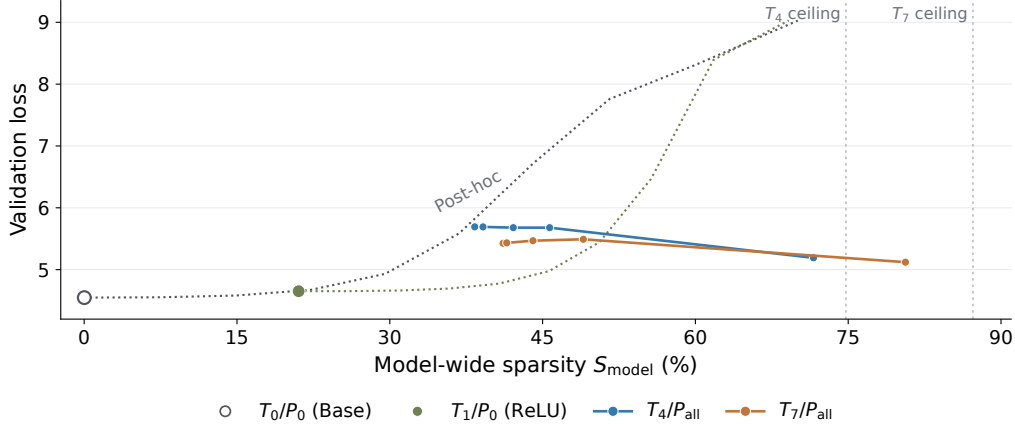


Figure 9: **410M quality-sparsity stress test.** All twelve trained conditions: Base, ReLU, and T_4/P_{all} and T_7/P_{all} at $\kappa = 0, .01, .05, .1, .5$. Dotted paths show all ten post-hoc targets for each fixed Base/ReLU control, including their full loss range. Vertical guides mark the T_4 and T_7 sparsity ceilings.

E KERNEL DEVELOPMENT AND EVALUATION

Implementation. The specialized forward pass preserves each trained model’s weights, thresholds, biases and residual connections. It combines CUDA, CUTLASS and FlashAttention components to fuse operations and skip matrix instructions whose activation operands are zero. It introduces no additional pruning. One kernel computes the two input LayerNorms from their shared input while retaining separate affine parameters and thresholding. QKV layout conversion, RoPE and the selected q, k, v gates are fused; q, k gates follow RoPE. The FFN-down and attention-output projections are evaluated jointly with their biases and residual addition. The final vocabulary projection remains dense. Table 10 specifies the sparse execution rules.

The scalar path requires finite weights and biases, with nonzero magnitudes between 2^{-50} and 2^{50} ; selected activations obey the same magnitude bounds. Other rows use the matrix path. The original

Table 10: **Sparse execution rules at 14M and in the original 70M port.** Tile dimensions are tokens by input features. A matrix multiply–accumulate (MMA) instruction operates on $16 \times 8 \times 16$ elements (output rows, output columns, reduction dimension). Zero checks and any scalar replacement execute during every forward pass.

Operation	Execution rule	Work avoided
QKV, FFN-up (a, m)	Test each 16×16 activation fragment after both operands have been loaded.	Skip the MMA for an empty fragment; retain operand reads.
FFN-down, attention output (h, z)	Inspect eight rows; use scalar arithmetic for rows with at most two nonzeros when numerical guards pass. Process the remaining rows in 8×16 tiles padded to 16 rows.	Skip empty tiles before loading their weights. Scalar rows read only the corresponding weights.
Attention (QK, PV)	After loading the operands, use a warp-wide check to test whether either complete MMA fragment is zero.	Skip that MMA; retain input loading, masking, softmax and output processing.

70M port retains these rules, with projection dimensions adapted from $(d, d_f) = (128, 512)$ to $(512, 2048)$. The 14M attention kernel uses 64×256 query/key tiles and two key/value partitions; the original 70M port uses 128×128 tiles without partitioning to match its native attention reduction order. This change was needed for numerical agreement, rather than selected by a 70M performance search.

Scope and numerical checks. Tests use one RTX 5090, BF16 inference, batch size one, 2,048-token causal sequences, and all 50,304 output logits, with PyTorch 2.11.0, Transformers 5.12.1 and CUDA 12.8. The implementation assumes fixed weights, shapes and non-overlapping use of its buffers. It is not evaluated for training, cached decoding, concurrent serving, arbitrary masks or thresholds outside the tested grid. Each final checkpoint must pass in three fresh processes on all 338 validation sequences. Relative to the unmodified eager implementation, outputs must be finite, every logit must satisfy $|y - \hat{y}| \leq 0.25 + 0.02|\hat{y}|$, the relative ℓ_2 error of each sequence’s logits must not exceed 0.02, and the absolute mean validation-loss difference must not exceed 0.001. These are numerical tolerances, not bitwise equivalence.

Timing and reference. Native PyTorch/SDPA and specialized inference use CUDA graphs with identical inputs. Latency is the geometric mean of host timings over 64 fixed validation sequences, seven randomized paired passes and three fresh processes. Compilation, static weight layout preparation, graph capture and equal input staging are excluded. Embeddings, all blocks, final normalization, vocabulary logits and input-dependent preprocessing are included. Post-hoc masks also remain inside timing; no activations are cached. The original 14M measurements and the new 70M sweep use separate GPU sessions. At 70M, native inference, the original port and the optimized implementation are paired in every process for all 26 original checkpoints. A later session repeats Base and T_2/P_h at $\kappa = 0.1$ and measures its additional $\kappa = 0.5$ checkpoint with the same frozen protocol (Table 7); absolute timings are not pooled across sessions.

For comparisons across recipes, we use *one native base-model latency per size and measurement session*: $t_{\text{native}, T_0/P_0} / t_{\text{specialized}, r}$ for recipe r . The native reference is 0.65544 ms for the retained 14M measurements; the new 70M reference is reported in Table 8. The specialized base-model latency is not used as the speedup denominator: an inefficient dense fallback can inflate that ratio. The Base marker in the 14M latency figure uses specialized execution; its native reference is reported here.

Adapting the kernel to 70M. An initial follow-up tested the base model and T_7/P_h at $\kappa = 0.5$. Replacing only h, z with native operations reduced the ported base-model latency by 1.661 ms, nearly its entire 1.674 ms deficit. This identifies the expensive component, not an additive breakdown of causes. Its general matrix path pads eight real rows to sixteen and, at width 512, repeats activation inspection across four output-column blocks rather than one at 14M. It also lacks a shared-memory weight pipeline. Sparse inputs can avoid much of this fallback: 94.88% of h rows and 99.59% of z rows have at most two nonzeros in the tested sparse checkpoint, versus none in the base model.

We selected the optimized implementation using sixteen fixed training blocks, then froze it before full-validation testing. It retains sparse h, z , fused normalization and RoPE, and native a, m projections. A row prepass identifies guarded scalar rows with at most eight nonzeros, accumulated in ascending feature order in FP32. Other rows retain the eight-row matrix path, with a 256-column output tile. All input-dependent inspection remains inside timing. Attention uses a dense tiled kernel without activation-induced MMA skipping, and the dense vocabulary projection uses a tuned CUTLASS kernel. These additional dense optimizations also contribute to latency reduction.

Table 11: **70M implementation comparison.** Representative rows from the complete 26-checkpoint sweep in Table 8. All timings are full-model geometric means from the same RTX 5090 session. Each implementation preserves the checkpoint and thresholds.

Checkpoint	Native (ms)	Port (ms)	Optimized (ms)	Native-base speedup
T_0/P_0	1.611	3.325	2.732	$0.590\times$
$T_2/P_h, \kappa = 0.1$	1.662	2.258	1.956	$0.824\times$
$T_4/P_h, \kappa = 0.5$	1.706	1.593	1.088	$1.480\times$
$T_7/P_h, \kappa = 0.5$	1.824	1.590	1.088	$1.481\times$

Table 11 compares representative checkpoints; Table 8 gives all 27 checkpoints, identifying the later $T_2/P_h, \kappa = 0.5$ measurement separately. Every implementation preserves the weights and thresholds and is checked on all 338 validation sequences in three fresh processes. This tests whether the observed sparse structures remain exploitable at 70M after implementation adaptation; it does not establish a scale-dependent speedup trend.

Dense optimizations and sparse execution. Retained controls from a separate optimization session distinguish these contributions. Replacing h, z with native dense operations while retaining the other optimized components reduces base-model latency from 1.701 to 1.297 ms, without activation sparsification. The optimized T_7/P_h model at $\kappa = 0.5$ takes 1.128 ms, 13.1% below that alternative dense base, but with higher loss. Replacing its sparse h, z implementation with native operations raises latency to 1.365 ms. This conditional difference includes fusion, layout and inspection costs; it is not a pure skip-toggle effect. Disabling skipping in the custom fallback instead gives 2.704 ms, illustrating why that inefficient path is not a practical dense reference. Component effects are conditional and non-additive, and these timings are not pooled with the new grid.

What MMA bypass measures. Bypass is the number of omitted MMA instructions divided by issued plus omitted instructions, pooling integer counts across layers and validation sequences. Projection totals pool the four projection operations; attention totals pool QK and PV . We keep the two groups separate because equal instruction counts need not imply equal costs. At h, z , bypass includes replacement by scalar arithmetic and counts padded rows; it is not a fraction of all arithmetic eliminated. Attention counts include causal padding; even the base model bypasses 10.42% of PV instructions at 14M. In contrast, $\mathcal{S}_{\text{model}}$ excludes masked pairs and includes the dense vocabulary projection in its denominator.

The figure’s projection scalar sparsity pools zero-activation products over all projection products in the actual BF16 operands. Both this measure and bypass cover all 338 validation sequences; timing uses the 64-sequence subset. Thus these diagnostics complement, rather than redefine, the canonical model-wide sparsity used in the quality figures.

Interpretation and controlled tests. Projection bypass tracks speedup at 14M, but high scalar sparsity alone need not produce empty instruction tiles. Post-hoc controls show this gap and incur separate masking costs. Attention bypass is nearly constant across the trained grid except for T_7/P_{all} at $\kappa = 0.5$; this single exception cannot establish a general attention-speedup relation. Bypass therefore describes exploitable structure without explaining all latency differences.

Post-hoc clipping reaches at most $1.025\times$ the native base-model speed at 14M; this small difference spans GPU sessions and does not establish a reliable clipping benefit. Clipping adds a separate masking pass, and the resulting zeros must still form groups the kernel can skip. Since fused clipping was not optimized, this characterizes the current execution path rather than a general limit of post-hoc sparsification.

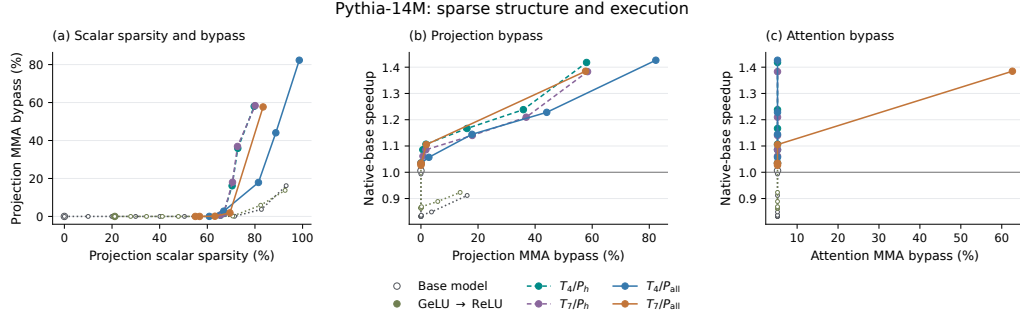


Figure 10: **Scalar sparsity, instruction bypass and speedup over a fixed native 14M base model.** The retained diagnostic covers 22 trained settings and 20 post-hoc control settings. Left: projection scalar sparsity versus projection MMA bypass. Middle/right: projection/attention bypass versus full-model speedup. Every speedup uses the same native T_0/P_0 reference of 0.65544 ms; the horizontal line is $1\times$. The Base model marker uses specialized inference, so it need not lie on that line. Lines connect measured thresholds, not fitted trends; dotted paths are post-hoc clipping. Counts are pooled, include the instruction-padding and scalar-substitution conventions above, and do not measure saved memory traffic.

Table 2 provides the direct 14M test: disable one skipping path while keeping weights, gates and fusion fixed. Its min-max spans cover all pairs of three independent off/on process means; effects are conditional and non-additive. These tests do not separately time memory reads, zero checks or softmax. The 70M follow-up above uses native component replacements, rather than this six-operation skip-toggle decomposition. The retained 14M scatter remains descriptive evidence, not causal attribution.

Alternatives tested. Porting the h, z strategy to a, m was correct but slower in the tested 14M case (Section 4.3). The padded fallback issued $1.90\times$ and $2.00\times$ as many matrix instructions, respectively. Attention shortcuts using cumulative value sums for zero queries, disjoint query/key supports, or cheaper conditional execution either failed numerical checks or did not establish a reliable gain. Disabling attention skipping improved all 35 historical 14M comparisons. The original grid implementation is therefore not latency-optimal; it preserves one common execution policy across recipes. The optimized 70M implementation instead uses dense attention. Earlier detection that avoids attention operand reads, fused post-hoc masking and sparsification of softmax probabilities remain untested here.